36. REST API Design, Middleware, Auth Basics

36. REST API Design, Middleware, Auth Basics

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `36-rest-middleware-auth`

Learning objectives

- Design resource-oriented REST endpoints
- Implement middleware for logging/auth
- Use bearer tokens / basic auth patterns
- Return consistent error JSON

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

REST middleware and auth are how you cross-cut logging, authn, and limits without copying code into every handler.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Middleware is `func(http.Handler) http.Handler`.
- Chain middleware outward-in carefully.
- Auth: validate tokens/headers early; set context values sparingly.
- Return proper status codes: 401/403/400 distinctions.
- Keep handlers thin: decode → validate → service → encode.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-rest && cd /tmp/gw-rest
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "net/http"
    "net/http/httptest"
)

func withHeader(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("X-Demo", "1")
        next.ServeHTTP(w, r)
    })
}

func main() {
    h := withHeader(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
        fmt.Fprint(w, "hi")
    }))
    rr := httptest.NewRecorder()
    h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/", nil))
    fmt.Println(rr.Header().Get("X-Demo"), rr.Body.String())
}
```

Run it:

```
go run .
```

Expected output:

```
1 hi
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write middleware that rejects missing Authorization.
2. Chain logging + auth middleware.
3. Map service errors to HTTP statuses.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

REST sketch

- `GET /items` list
- `GET /items/{id}` get
- `POST /items` create
- `DELETE /items/{id}` delete

Use proper status codes: 201 create, 404 missing, 400 validation, 401/403 auth.

Middleware

```
func WithRequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-ID")
        if id == "" { id = uuidNew() }
        ctx := context.WithValue(r.Context(), ctxKeyID, id)
        w.Header().Set("X-Request-ID", id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

Auth basics

Validate `Authorization` header; never roll crypto casually — use proven libraries. Separate authentication (who) from authorization (what allowed).

Failure modes to watch while coding

- Leaking stack traces to clients
- Auth only on some routes by accident
- Using GET for state changes

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Decorators ≈ middleware.
- Flask before_request hooks ≈ middleware chains.
- Keep crypto out of handlers — use small auth packages.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Prefer explicit dependency injection of auth services over global state.

Practice checklist

Before the exercise, confirm you can:

1. Explain “REST API Design, Middleware, Auth Basics” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Leaking stack traces to clients
- Auth only on some routes by accident
- Using GET for state changes

Exercises

Work in `exercises/36-rest-middleware-auth/`. Read that folder's `README.md` before you code.

What you will build

Implement middleware that enforces headers/auth as tests describe.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/36-rest-middleware-auth/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. 401 vs 403?
2. What does middleware do?
3. Why consistent error shape?

Further reading

- https://go.dev/doc/tutorial/web-service-gin